

Resizable BAR & D3D12 GPU Upload Heaps

유영천

<https://megayuchi.com>

DirectX 12 업데이트를 통해 CPU와 GPU가 동시에 VRAM에 액세스할 수 있습니다.

소식 By 아론 클로츠 2023년 4월 1일 게시

이 기능은 비디오 게임 RAM 사용률과 CPU 사용률을 줄일 수 있습니다.



당사 사이트의 링크를 통해 구매하면 제휴 수수료를 받을 수 있습니다. 작동 방식은 다음과 같습니다.

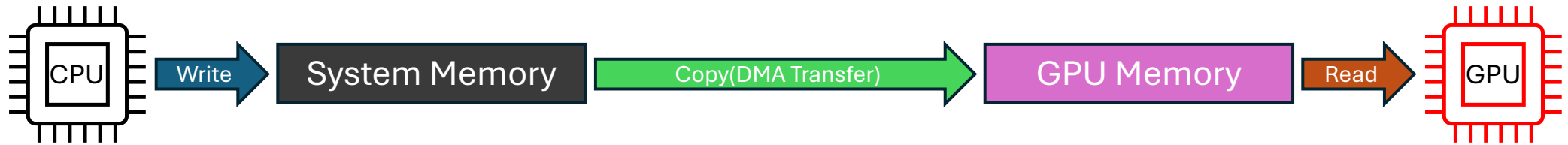
Microsoft는 CPU가 GPU 메모리에 직접 동시에 액세스할 수 있도록 하는 GPU [업로드 힙](#)이라는 Resizable-BAR와 함께 새로운 DirectX12 GPU 최적화 기능을 발표했습니다. 이렇게 하면 DX12 타이틀의 성능이 향상되고 이 기능이 CPU에서 GPU로 데이터를 복사할 필요가 없기 때문에 시스템 RAM 사용률을 줄일 수 있습니다. 새로운 기능은 현재 Agility SDK에서 사용할 수 있습니다.

이 기능의 실제 의미는 알 수 없지만 성능 이점은 상당할 수 있습니다. 그래픽 카드 메모리 크기와 비디오 게임 VRAM 소비는 매년 점점 더 커지고 있습니다. 결과적으로 CPU는 자신과 GPU 간에 점점 더 많은 데이터를 이동해야 합니다.

이 기능을 사용하면 데이터 전송 감소만으로도 게임의 RAM 및 CPU 사용률이 눈에 띄게 감소할 수 있습니다. 이는 CPU가 더 이상 상호 작용하기 위해 시스템 RAM과 GPU VRAM 모두에 데이터 복사본을 보관할 필요가 없기 때문입니다. 또 다른 보너스는 요즘 GPU 비디오 메모리가 매우 빠르기 때문에 GPU에만 데이터를 남겨두는 것에 대한 대기 시간 페널티가 없어야 한다는 것입니다. 실제로 고속 비디오 메모리가 있는 고급 GPU에서 CPU 액세스 시간으로 인해 대기 시간이 개선될 것입니다.

GPU에서 리소스를 소비하는 방법

CPU -> System Memory -> GPU Memory -> GPU에서 소비 (VertexBuffer / ConstantBuffer / Texture)



CPU -> System Memory -> GPU에서 소비 (VertexBuffer / ConstantBuffer)



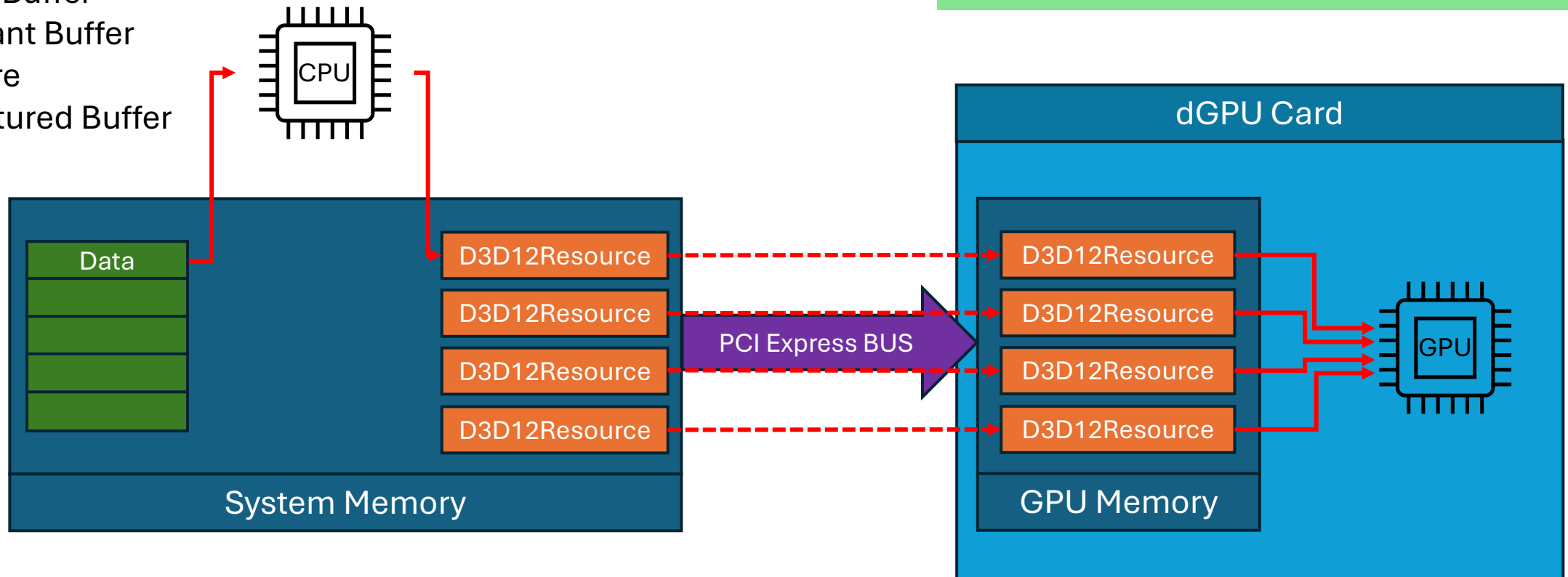
CPU -> GPU Memory -> GPU에서 소비 (VertexBuffer / ConstantBuffer / Texture)



Asynchronous DMA Transfer(PCI Burst Memory)

Vertex Buffer
Index Buffer
Constant Buffer
Texture
Structured Buffer

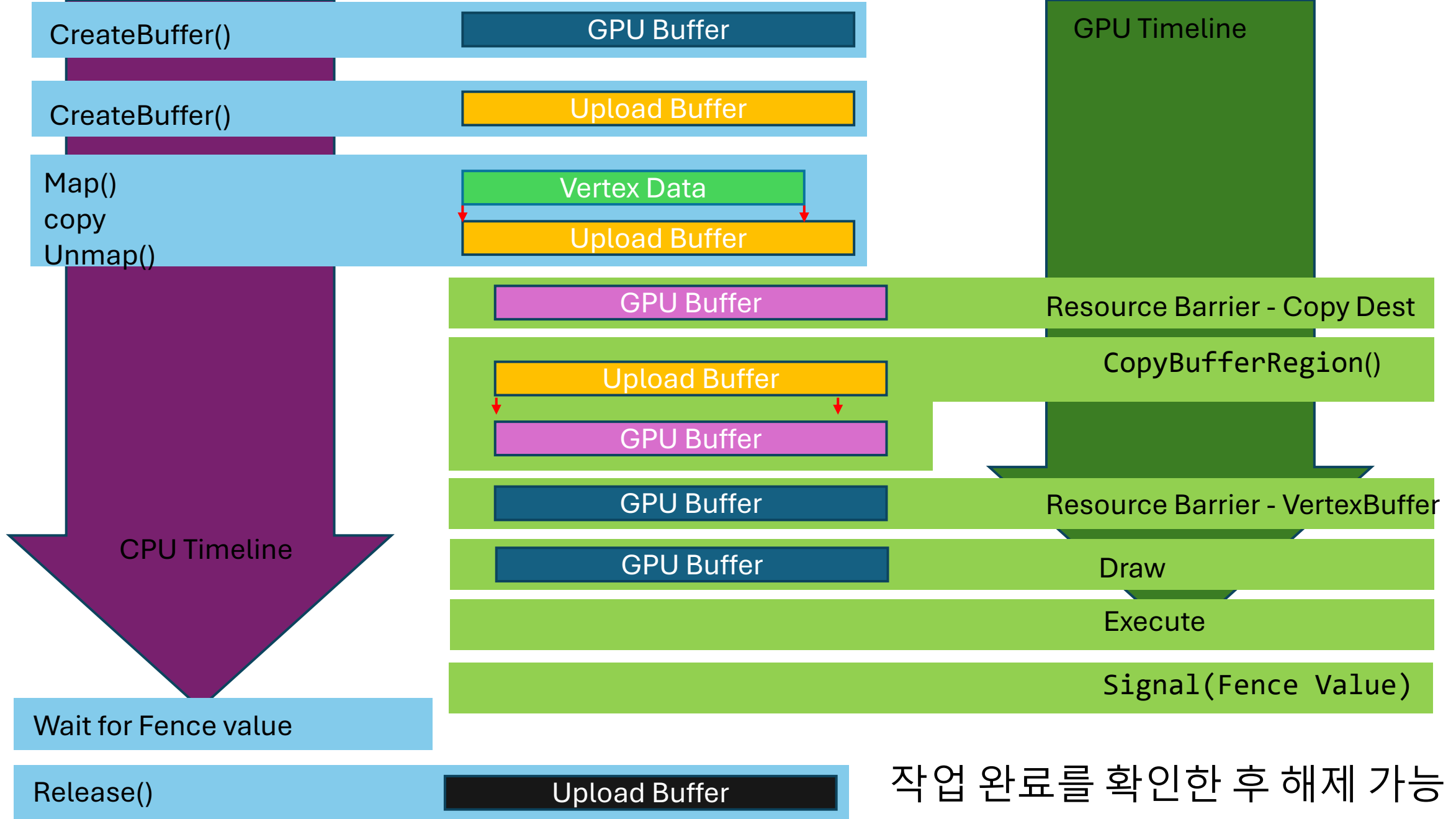
실행 주체가 CPU가 아닌 GPU임.
GPU에서 System Memory → GPU Memory
전송을 수행.



Copy data
System Memory → GPU Memory

Asynchronous DMA Transfer(PCI Burst Memory)

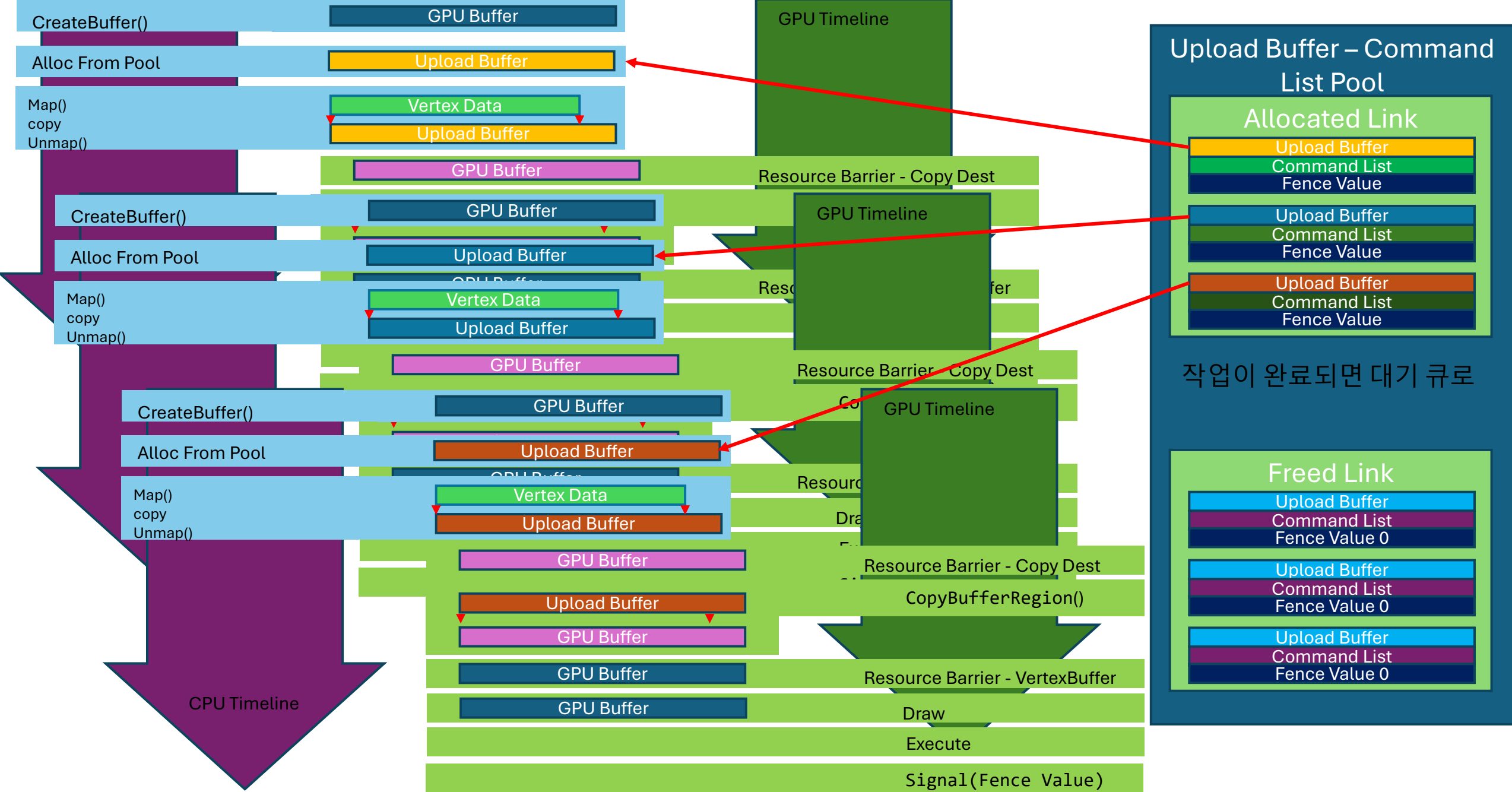
- 리소스가 시스템 메모리에 있음.
- 리소스를 GPU에 올려놓기 위해 CPU가 시스템 메모리에 데이터를 써넣고 GPU에게 이를 복사해가라고 명령(driver를 통해서).
- GPU는 비동기적으로 copy.
- GPU리소스에 데이터를 업데이트하는 기본적인 방법.
- GPU에서 리소스를 소비할 때의 성능 가장 좋음.
- 모든 리소스 타입에 대해 사용 가능.



단순 구현 - 전송 요청 -> fence -> wait

Asynchronous DMA Transfer(PCI Burst Memory)

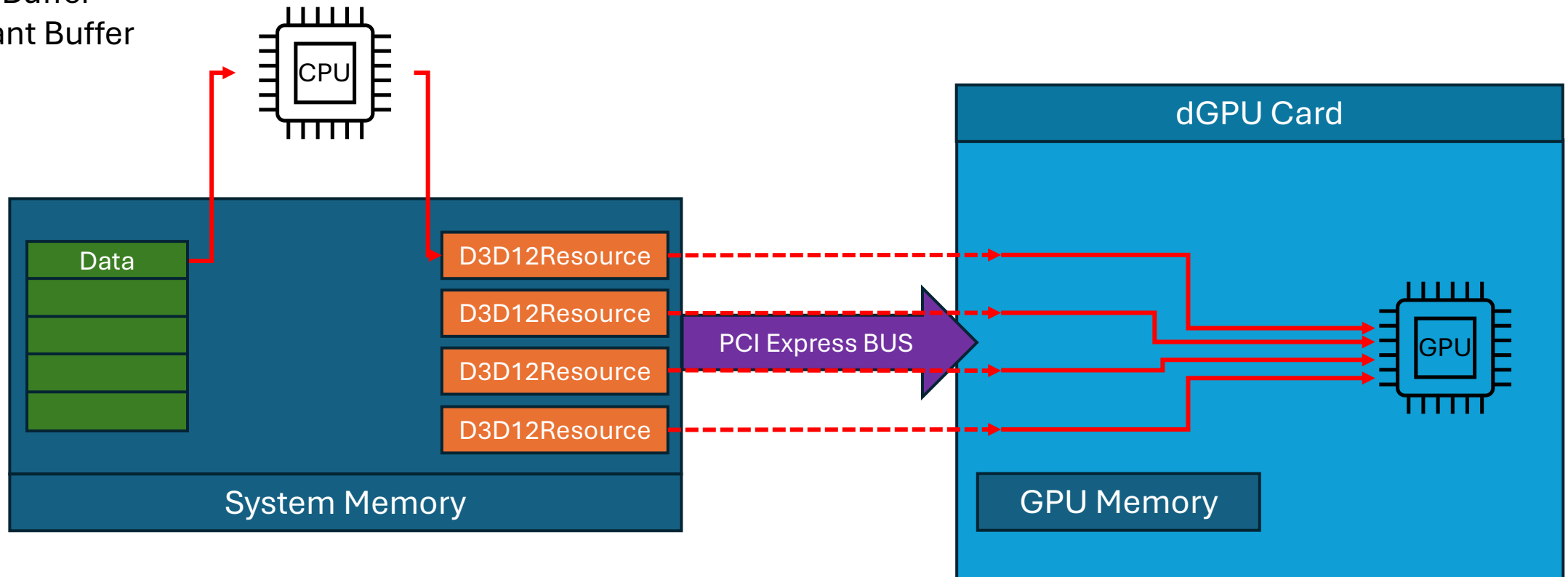
- Copy가 완료 되어야 소비(렌더링)이 가능. 따라서 동기화를 위해 resource barrier가 필요.
- Copy가 완료되면 copy를 위해 사용했던 시스템 메모리를 해제해야함. 하지만 copy가 언제 끝나는데???
- Copy작업이 끝났음을 대기 <- 엄청난 성능 저하.
 - 단순 구현 - 전송 요청 -> fence -> wait : 엄청나게 느리다.
 - 실제 사용을 위해서는 복잡한 비동기 업데이트 매니저 코드가 필요.
 - 추가 코드 잔뜩 붙음.



비동기 업데이트 매니저 구현(D3D12렌더러 구현할 때 필수)

GPUMMU or GART (Graphics Address Remapping Table)

Vertex Buffer
Index Buffer
Constant Buffer



Zero copy

GPUMMU or GART (Graphics Address Remapping Table)

- 시스템 메모리에 리소스를 올려두고 GPU가 이를 직접 사용.
- 추가 코드 없이 CPU가 직접 시스템 메모리에 데이터를 써넣으므로 업데이트 빠름.
- GPU에서 리소스를 소비할 때의 성능이 떨어짐.
- 간결한 코드.
- 성능 문제와 구조적 문제로 Texture, Structured Buffer에 대해 사용 불가.

CPU -> GPU Memory -> GPU에서 소비

- CPU에서 직접 GPU메모리에 써넣으면 안되나?
- UMA(CPU/GPU 통합 메모리 아키텍처)에선 가능함.
 - Apple silicon, 게임기들, intel / AMD 내장 GPU들...
- PCIe 슬롯에 꽂는 dGPU들의 경우 간단하지 않음.
 - CPU와 GPU의 주소체계가 다르다.
 - CPU와 GPU 각각 다른 MMU를 가진다.
 - CPU -> GPU Memory는 Memory operation이 아닌 I/O operation임.

CPU가 주변장치와 통신하는 방법

- PMIO(Port-Mapped I/O)
 - I/O전용의 별도 주소공간 사용(x86의 경우 16 bits주소공간).
 - In/out 명령어를 사용해서 장치에 대해 데이터를 읽고 씀.
 - 제어를 위해 주로사용. 대량의 데이터를 주고 받는 용도로는 적합하지 않음.
- MMIO(Memory-Mapped I/O)
 - 장치의 레지스터 or 주소공간을 CPU의 메모리 주소공간에 맵핑.
 - I/O port에 비해 훨씬 큰 주소공간 사용 가능.
 - I/O를 통한 write는 느리지만 Write Combining기법을 사용해서 성능 향상.

BAR (Base Address Register) Mapping - MMIO

- PCI이전 세대
 - 장치들의 MMIO주소가 고정(반고정)이었음.
 - 장치 추가 시 주소충돌 문제.
- PCI와 현대 OS의 시대 이후
 - BIOS에서 장치마다 동적으로 MMIO주소 할당.BAR레지스터에 설정.
 - OS가 장치의 BAR 레지스터를 읽어서 MMIO주소로 사용.
 - 장치의 추가 및 성능향상에 대해 유연하게 대처할 수 있음.

BAR를 이용한 MMIO주소 할당

1. 시스템 전원 ON.
2. BIOS 가 PCIe 버스를 트리 구조로 탐색.
3. BIOS는 PCIe 장치의 BAR 레지스터를 읽어서 장치가 요구하는 메모리 공간 크기를 확인.
4. BIOS는 충돌없이 MMIO주소를 할당
 - BAR레지스터에 시작 주소 위치와 사이즈 기록.
5. OS부팅시 OS에서 BAR레지스터를 읽어서 사용.
6. 장치별 주소공간의 최대 크기는 256MB로 제한.

PCI Configuration Space

31	16 15				0
Device ID		Vendor ID			00h
Status		Command			04h
Class Code			Revision ID		08h
BIST	Header Type	Lat. Timer	Cache Line S.		0Ch
Base Address Registers					10h
					14h
					18h
					1Ch
					20h
					24h
					28h
Cardbus CIS Pointer					28h
Subsystem ID		Subsystem Vendor ID			2Ch
Expansion ROM Base Address					30h
Reserved			Cap. Pointer		34h
Reserved					38h
Max Lat.	Min Gnt.	Interrupt Pin	Interrupt Line		3Ch

- Vendor ID / Device ID: 장치 제조사 및 모델 ID
- Command / Status Register: 장치 활성화 및 상태 확인
- Class Code: 장치 종류 (예: VGA, 네트워크 카드 등)
- Header Type: 장치 유형 (Endpoint, Bridge 등)
- BAR Registers (BAR0~BAR5): 장치가 사용할 메모리 또는 I/O 공간의 시작 주소와 크기 지정

장치의 MMIO 메모리 주소공간 크기를 얻고 시작주소를 설정

```
; bus = 1, device = 0, function = 2, offset(bar0) = 0x10
; 0x80000000 | bus << 16 | device << 11 | function << 8 | offset
; CONFIG_ADDRESS = 0x80011010 (bus=1, dev=0, func=2, offset=0x10)
```

```
; query memory size for the device
; Write 0xFFFFFFFF to BAR0
mov dx, 0CF8h          ; port address for PCI_CONFIG_ADDRESS
mov eax, 080011010h    ; PCI_CONFIG_ADDRESS
out dx, eax            ; Set PCI_CONFIG_ADDRESS
```

```
mov dx, 0CFCh          ; port address for PCI_CONFIG_DATA
mov eax, 0FFFFFFFFh    ; filled value
out dx, eax            ; write to bar0 with 0FFFFFFFFh
```

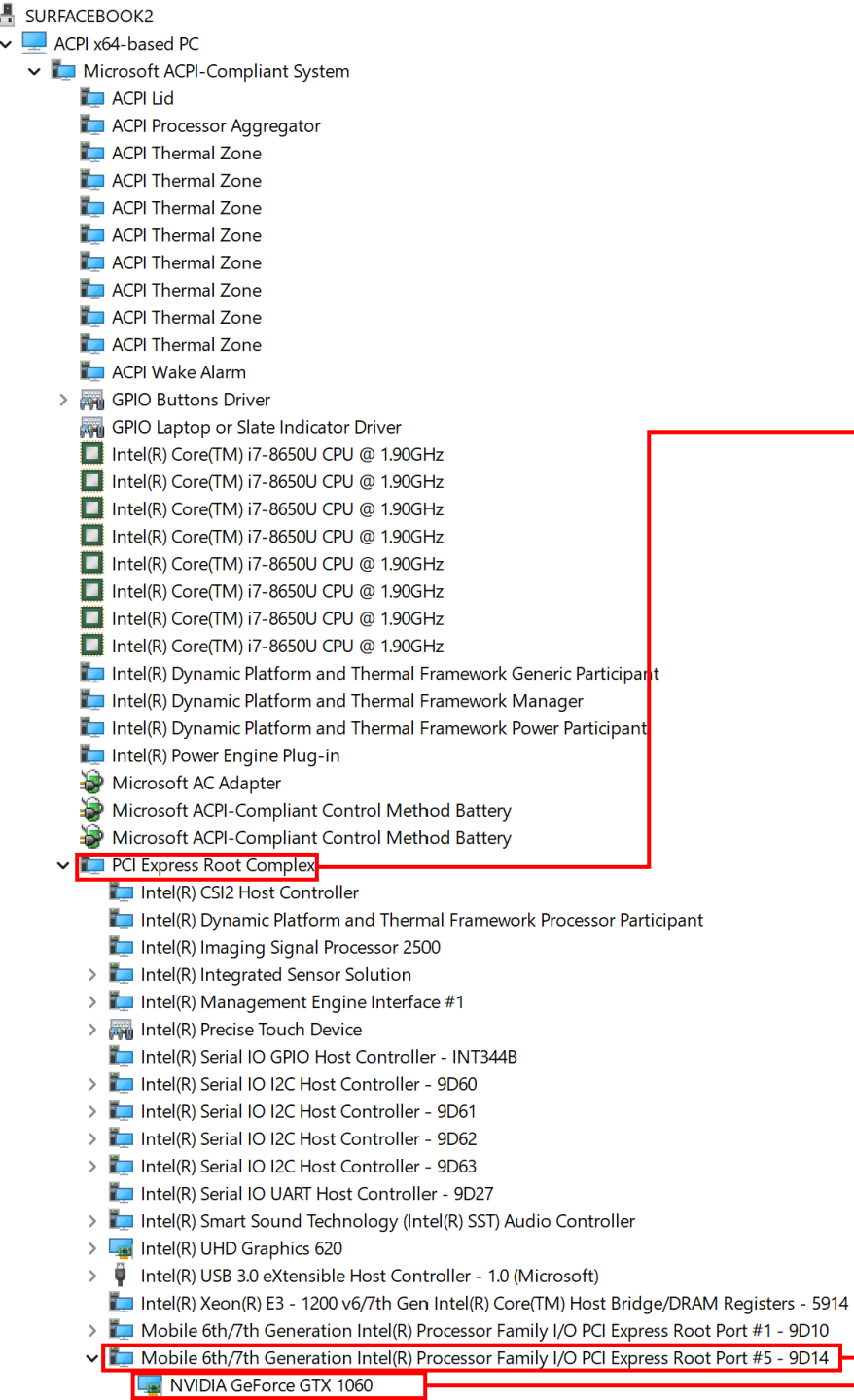
```
; Read back BAR0 mask value
mov dx, 0CF8h          ; port address for PCI_CONFIG_ADDRESS
mov eax, 080011010h    ; PCI_CONFIG_ADDRESS
out dx, eax            ; Set PCI_CONFIG_ADDRESS
```

```
mov dx, 0CFCh ; port address for PCI_CONFIG_DATA
in  eax, dx    ; read bar0
```

```
; calculate size , if bar0 = 0xffff0000 , 0xffff0000 -> 0x000fffff
; 0x000fffff + 1 = 0x00100000 -> 1MB
and eax, 0FFFFFFF0h    ; mask lower 4 bits (bit 0~3)
not eax
inc eax
mov bar_size, eax      ; save size of MMIO-address-space
```

```
; write MMIO base address: 0xFE000000
mov dx, 0CF8h          ; port address for PCI_CONFIG_ADDRESS
mov eax, 080011010h    ; CONFIG_ADDRESS
out dx, eax            ; set PCI_CONFIG_ADDRESS
```

```
mov dx, 0CFCh          ; port address for PCI_CONFIG_DATA
mov eax, 0FE000000h    ; MMIO base address
out dx, eax            ; write new BAR0 value
```

PCI Express Root Complex

- IO : 0000-0cf7
- IO : 0d00-ffff
- MEM : 000a0000-000bffff
- MEM : 000c0000-000c3fff
- MEM : 000c4000-000c7fff
- MEM : 000c8000-000cbfff
- MEM : 000cc000-000cffff
- MEM : 000d0000-000d3fff
- MEM : 000d4000-000d7fff
- MEM : 000d8000-000dbfff
- MEM : 000dc000-000dffff
- MEM : 000e0000-000e3fff
- MEM : 000e4000-000e7fff
- MEM : 000e8000-000ebfff
- MEM : 000ec000-000effff
- MEM : 000f0000-000fffff
- MEM : 8f800000-dfffffff
- MEM : fd000000-fe7fffff

PCI Express Root Port #5 - 9D14

- MEM : c1000000-c1ffffff
- MEM : 90000000-a1ffffff
- IO : 3000-3fff
- IRQ : 4294967276

NVIDIA GeForce GTX 1060

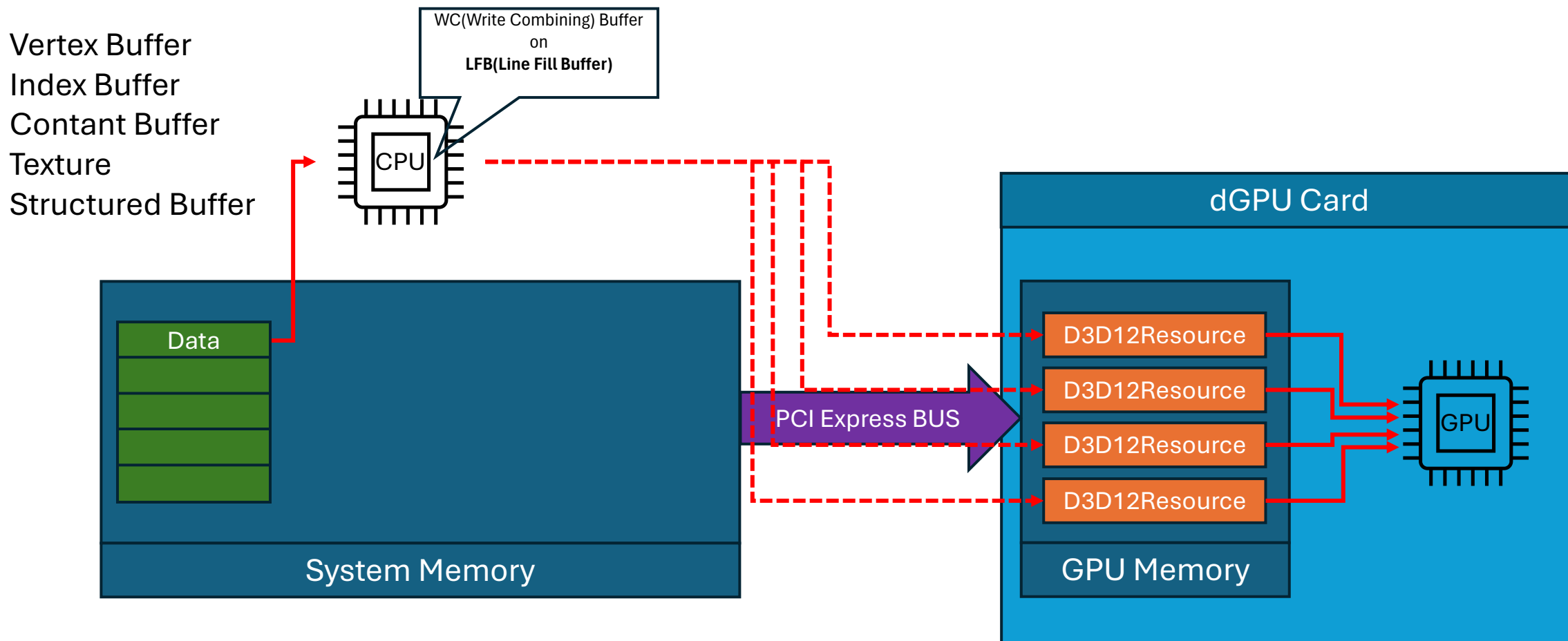
- MEM : c1000000-c1ffffff
- MEM : 90000000-9fffffff
- MEM : a0000000-a1ffffff
- IRQ : 4294967275

Root Complex/Root Port가 TLP를 하위 PCI장치로 라우팅 하기 위해 PCI장치의 MMIO주소는 Root Complex의 MMIO Aperture에 포함.

BAR-MMIO를 이용해서 GPU Memory에 쓰기

- MMIO를 이용해서 CPU가 GPU메모리에 직접 데이터를 써넣음.
- MMIO의 구조적 한계로 인한 쓰기 성능 저하 가능성 있으나 쓰기 성능 저하는 Write Combining기법으로 보완.

MMIO를 이용한 D3D Resource 사용



Zero copy

BAR-MMIO를 이용해서 GPU Memory에 쓰기

- 하지만 게임 리소스는 수십GB에 달한다 - 256MB제약으로 현대적인 리얼타임 그래픽 API에서는 사용하기엔 무리다.

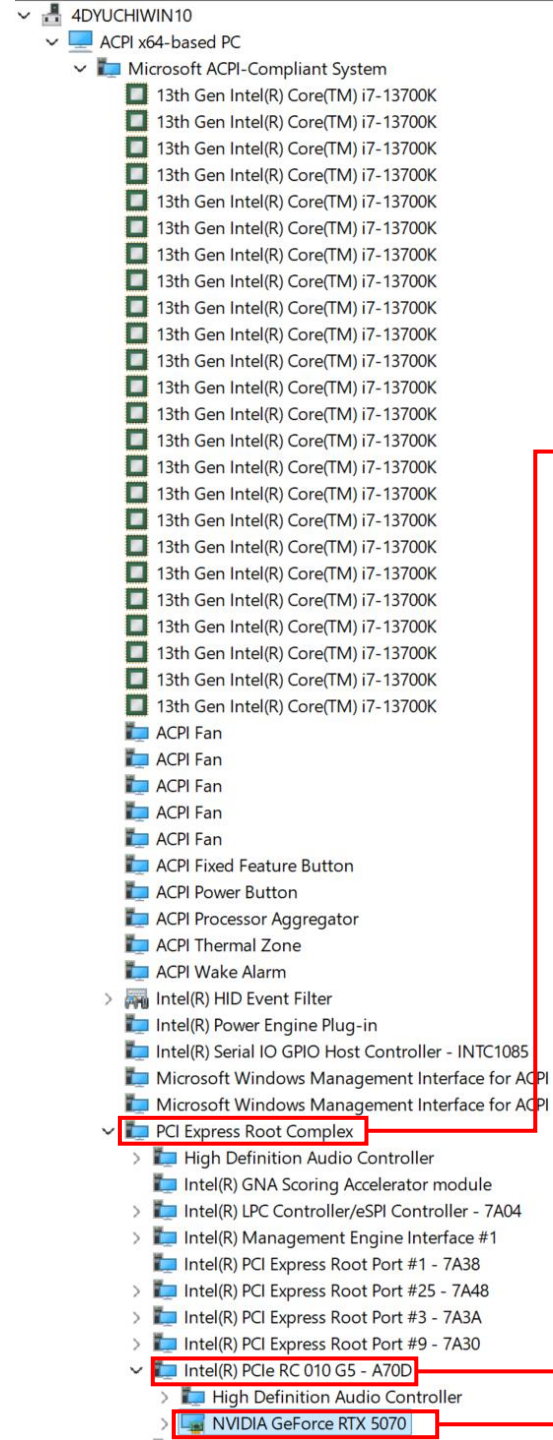
D3D는 Resource 소비 통로로 MMIO를 사용하는가?

- DirectDraw에선 전체화면모드에서 대해 (넓은 의미에서의 MMIO)를 사용 했었음.
- D3D에서도 어쩌면 어느 버전까진 사용했을 수도???
- 적어도 D3D12에선 GPU Memory에 올려둔 D3DResource를 CPU가 액세스할 수 없었음 - D3D12에선 안(못)썼어요.
- 장치당 256MB의 주소공간 사용가능 - 256MB 주소공간은 게임의 리소스를 맵핑하기엔 너무나 부족함.

Resizable BAR

- BAR의 크기를 동적으로 조정할 수 있는 PCIe 기능 확장.
- OS는 Resizable BAR 기능이 활성화되어 있으면 장치에 대한 BAR 크기를 재협상
- GPU의 메모리 전체를 CPU의 연속된 메모리 공간으로 맵핑 가능.
 - Windows - WDDM 2.0 이상.
 - PCI-E 3.0이상.
 - GPU와 메인보드가 지원해야 함.
 - nvidia 기준 RTX3000이상부터 지원.

Resizable BAR enabled



• PCI Express Root Complex

- IO : 0000-0cf7
- IO : 0d00-ffff
- MEM : 000a0000-000bffff
- MEM : 000e0000-000e3fff
- MEM : 000e4000-000e7fff
- MEM : 000e8000-000ebfff
- MEM : 000ec000-000effff
- MEM : 000f0000-000fffff
- MEM : 40400000-bfffffff : 2GB

• Intel(R) PCIe RC 010 G5 - A70D

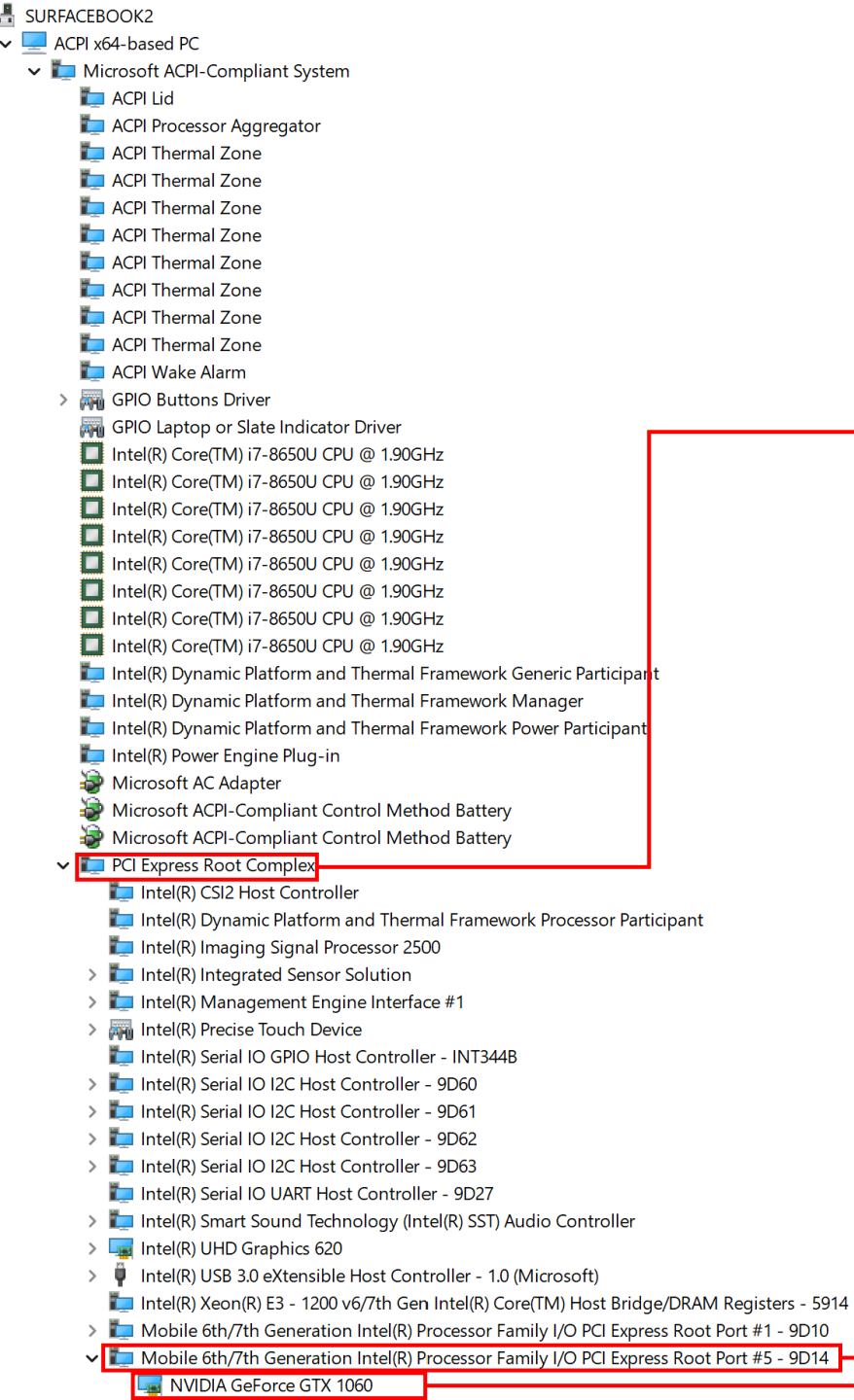
- MEM : 44000000-481fffff : 65MB
- IO : 5000-5fff
- IRQ : 4294967294

• NVIDIA GeForce RTX 5070

- MEM : 44000000-47ffffff : 63MB
- MEM : 6412000000-6413ffffff : 31MB
- MEM : 6000000000-63ffffff : 16GB
- IO : 5000-507f
- IRQ : 4294967228

Root Complex/Root Port가 TLP를 하위 PCI장치로 라우팅 하기 위해 PCI장치의 MMIO주소는 Root Complex의 MMIO Aperture에 포함.

Resizable BAR로 할당된 주소공간
Root Complex/Root Port의 주소공간 바깥



PCI Express Root Complex

- IO : 0000-0cf7
- IO : 0d00-ffff
- MEM : 000a0000-000bffff
- MEM : 000c0000-000c3fff
- MEM : 000c4000-000c7fff
- MEM : 000c8000-000cbfff
- MEM : 000cc000-000cffff
- MEM : 000d0000-000d3fff
- MEM : 000d4000-000d7fff
- MEM : 000d8000-000dbfff
- MEM : 000dc000-000dffff
- MEM : 000e0000-000e3fff
- MEM : 000e4000-000e7fff
- MEM : 000e8000-000ebfff
- MEM : 000ec000-000effff
- MEM : 000f0000-000fffff
- MEM : 8f800000-dfffffff
- MEM : fd000000-fe7fffff

PCI Express Root Port #5 - 9D14

- MEM : c1000000-c1ffffff
- MEM : 90000000-a1ffffff
- IO : 3000-3fff
- IRQ : 4294967276

NVIDIA GeForce GTX 1060

- MEM : c1000000-c1ffffff
- MEM : 90000000-9fffffff
- MEM : a0000000-a1ffffff
- IRQ : 4294967275

Resizable BAR disabled

Root Complex/Root Port가 TLP를 하위 PCI장치로 라우팅 하기 위해 PCI장치의 MMIO주소는 Root Complex의 MMIO Aperture에 포함.

Resizable BAR로 할당된 주소공간 없음.

GPU Upload Heaps

- Resizable BAR를 이용하여 GPU Memory 전체를 MMIO 주소 공간으로 맵핑 가능.
- MMIO로 맵핑된 GPU Memory를 D3D12 Resource로 사용.
- CPU가 직접 읽고 쓰기 가능(읽기는 가능한 하지만 성능상의 이유로 권장되지 않음).
- 내부적으로 PCI-E 버스를 통한 전송이므로 system memory 쓰기보다 느리지만 Write combining Buffer를 사용할 수 있으므로 쓰기에 있어서는 성능저하가 적다.
- 비동기 전송을 통한 방식에 비해 copy 작업이 없고 전송준비/전송확인 코드가 필요없어 성능향상을 기대할 수 있다.
- 코드가 간결해짐(개발/유지보수 용이->프로그래머가 편해짐).

GPU Upload Heaps - 배경

<https://microsoft.github.io/DirectX-Specs/d3d/D3D12GPUUploadHeaps.html>

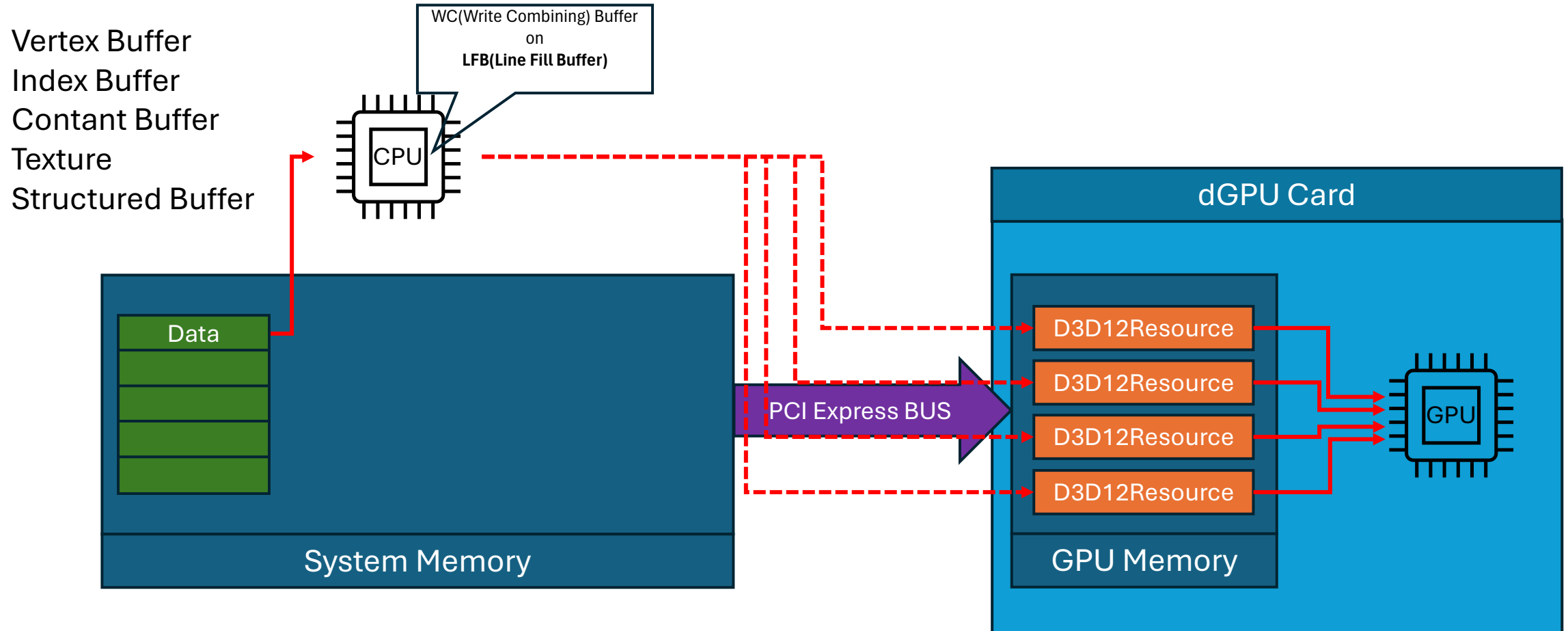
개별 GPU는 프레임 버퍼의 작은 부분만 PCI 버스를 통해 노출되는 것이 일반적이었습니다. D3D는 프레임 버퍼의 I/O 영역이 일반적으로 256MB에 불과하므로 유용하지 않기 때문에 이를 노출하지 않기로 결정했습니다. 더 큰 문제는 적절하게 가상화 가능한 리소스가 아니라는 것입니다. 앱 A가 들어와 256MB를 모두 할당하면 앱 B는 아무것도 할당할 수 없습니다. 따라서 우리는 그것에 대해 어떠한 보장도 제공할 수 없으며, 그 시점에서 앱에 노출하는 것은 나쁜 생각처럼 보였습니다.

그러나 이제는 크기 조정 가능한 기본 주소 레지스터(크기 조정 가능한 BAR라고도 함)를 지원하는 GPU가 많이 있습니다. Resizable BAR를 사용하면 Windows는 WDDM 버전 2.0 이상에서 GPU의 BAR 크기를 다시 협상합니다. D3D12가 크기 조정 가능한 BAR를 노출하는 또 다른 좋은 이유는 일부 플랫폼에서 일부 성능 테스트에서 크기 조정 가능한 BAR를 사용하도록 설정하면 게임이 더 나은 성능을 발휘하는 것으로 나타났기 때문입니다.

D3D는 이제 개별 GPU 및 통합 GPU 모두에서 D3D GPU 업로드 힙을 통해 CPU 표시 VRAM 사용을 지원하기로 결정했습니다.

통합 GPU에서 GPU 업로드 힙을 사용하는 한 가지 이유는 리소스에 업로드 + 기본 힙을 사용하는 대신 GPU 업로드 힙을 사용할 수 있으므로 리소스를 다른 상태로 전환하기 위해 많은 복사 작업을 수행할 필요가 없기 때문입니다. 또는 UMA 최적화: CPU 액세스 가능한 텍스처 및 표준 스위즐을 통해 이 최적화를 수행할 수 있습니다.

GPU Upload heaps를 이용한 D3D Resource 사용



Zero copy

Vertex Buffer / Index Buffer / Constant Buffer / Structured Buffer

```
ID3D12Resource* pVertexBuffer = nullptr;
UINT VertexBufferSize = SizePerVertex * dwVertexNum;

if (FAILED(pDevice->CreateCommittedResource(
    &CD3DX12_HEAP_PROPERTIES(D3D12_HEAP_TYPE_GPU_UPLOAD),
    D3D12_HEAP_FLAG_NONE,
    &CD3DX12_RESOURCE_DESC::Buffer(VertexBufferSize),
    D3D12_RESOURCE_STATE_COMMON,
    nullptr,
    IID_PPV_ARGS(&pVertexBuffer)))
{
    __debugbreak();
}

CD3DX12_RANGE readRange(0, 0);
char* pDest = nullptr;
pVertexBuffer->Map(0, &readRange, (void**)&pDest);
memcpy(pDest, pSrcData, VertexBufferSize);
pVertexBuffer->Unmap(0, nullptr);
```

Texture

```
// Create Texture for Render
if (FAILED(pD3DDevice->CreateCommittedResource(
    &CD3DX12_HEAP_PROPERTIES(D3D12_HEAP_TYPE_GPU_UPLOAD),
    D3D12_HEAP_FLAG_NONE,
    &textureDesc,
    D3D12_RESOURCE_STATE_ALL_SHADER_RESOURCE,
    nullptr,
    IID_PPV_ARGS(&pTexResource))))
{
    __debugbreak();
}

D3D12_SUBRESOURCE_DATA textureData = {};
textureData.pData = temp.get();
textureData.RowPitch = rowPitch;
textureData.SlicePitch = textureData.RowPitch * theight;

pTexResource->WriteToSubresource(0, nullptr, textureData.pData, textureData.RowPitch,
textureData.SlicePitch);
```

Texture 데이터는 D3D Resource 상에서 선형적으로 배치되지 않으므로 선형적으로 배열된 픽셀 데이터를 D3D Resource의 메모리에 직접 copy해서는 안된다.

Update 방법 비교

1. [fence/wait] CPU->UploadBuffer(System)->DMA Copy->GPU-Resource -> fence/wait
 - 예제에서나 사용하는 정도. 또는 초기화 코드에서만 사용. 실전에서 사용하기엔 성능이 너무 떨어짐.
2. [no wait] CPU->UploadBuffer(System)->DMA Copy(async update manager)->GPU-Resource
 - 1에 비해서는 압도적인 성능. GPU Upload Heaps feature가 지원되지 않는 장비에선 선택의 여지가 없다. 무조건 구현해야 함.
3. [no wait] CPU->UploadBuffer(GPU Upload Heaps)
 - 2에 비해서 5%-30% 성능 향상이 있음. 코드가 간결해져서 이쪽으로 통일하면 좋겠지만 지원되지 않는 장비가 여전히 많음. 옵션으로 지원.

아키텍처에 따른 D3D12 Resource의 메모리 속성

- UMA지원 : GPU/CPU 통합메모리 시스템
- UMA장치에서 CacheCoherentUMA 지원 :
 - CPU와 GPU 간 메모리 액세스 시 캐시 동기화가 자동으로 유지됨.
- MemoryPoolPreference
 - L0 : System Memory
 - L1 : GPU Memory
- CPUPageProperty
 - WRITE_BACK
 - 일반적인 캐시 정책. 캐시에 먼저쓰고 나중(필요한 때)에 메모리에 써넣기.
 - 읽기/쓰기/랜덤액세스에서 적당히 좋은 성능.
 - Cache coherency보장됨
 - WRITE_COMBINE
 - WC Buffer에 데이터를 모아서(일반적으로 64 bytes) 일괄적으로 써넣기.
 - 연속된 데이터일 때 쓰기 매우 빠름/읽기 많이 느림.
 - Cache coherency보장 안됨

아키텍처에 따른 D3D12 Resource의 메모리 속성

D3D12_FEATURE_DATA_ARCHITECTURE::UMA == FALSE

Heap Type	How the returned D3D12_HEAP_PROPERTIES members convert
D3D12_HEAP_TYPE_UPLOAD	CPUPageProperty = WRITE_COMBINE, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_DEFAULT	CPUPageProperty = NOT_AVAILABLE, MemoryPoolPreference = L1.
D3D12_HEAP_TYPE_READBACK	CPUPageProperty = WRITE_BACK, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_GPU_UPLOAD	CPUPageProperty = WRITE_COMBINE, MemoryPoolPreference = L1.

아키텍처에 따른 D3D12 Resource의 메모리 속성

D3D12_FEATURE_DATA_ARCHITECTURE::UMA == TRUE

D3D12_FEATURE_DATA_ARCHITECTURE::CacheCoherentUMA == FALSE

Heap Type	How the returned D3D12_HEAP_PROPERTIES members convert
D3D12_HEAP_TYPE_UPLOAD	CPUPageProperty = WRITE_COMBINE, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_DEFAULT	CPUPageProperty = NOT_AVAILABLE, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_READBACK	CPUPageProperty = WRITE_BACK, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_GPU_UPLOAD	CPUPageProperty = WRITE_COMBINE, MemoryPoolPreference = L0.

아키텍처에 따른 D3D12 Resource의 메모리 속성

D3D12_FEATURE_DATA_ARCHITECTURE::UMA == TRUE

D3D12_FEATURE_DATA_ARCHITECTURE::CacheCoherentUMA == TRUE

Heap Type	How the returned D3D12_HEAP_PROPERTIES members convert
D3D12_HEAP_TYPE_UPLOAD	CPUPageProperty = WRITE_BACK, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_DEFAULT	CPUPageProperty = NOT_AVAILABLE, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_READBACK	CPUPageProperty = WRITE_BACK, MemoryPoolPreference = L0.
D3D12_HEAP_TYPE_GPU_UPLOAD	CPUPageProperty = WRITE_BACK, MemoryPoolPreference = L0.

프로그래밍시 주의

- GPU가 액세스 중인 리소스에 즉시 써넣을 수 있으므로 동시에 2프레임 이상 렌더링 하는 경우 리소스를 2중화(더블 버퍼링) 시켜야 함.

예제분석

- <https://github.com/megayuchi/D3D12Lecture>
 - 23번 예제

성능비교(VOXEL HORIZON)- 맵 로딩 시간 비교

	Name	CPU	Working set (memory)	Memory (active private working set)	Paged pool	Threads	Platform	GPU	GPU engine	Dedicated GPU memory	Shared GPU memory	Loading Time(ms)
[D3D12]												
Map-V101												
GPU Upload Heaps(release)	Client_x64_release.exe	3	1,512,508 K	1,434,972 K	2,116 K	211	64 bit	34	GPU 1 - 3D	826,240 K	477,400 K (-11%)	725.1 (-10%)
UpdateResource(release)	Client_x64_release.exe	3	1,586,492 K	1,509,016 K	2,179 K	212	64 bit	35	GPU 1 - 3D	827,656 K	537,176 K	800.75
Map-V111												
GPU Upload Heaps(release)	Client_x64_release.exe	14	2,273,780 K	2,196,240 K	2,863 K	212	64 bit	46	GPU 1 - 3D	999,320 K	655,680 K (-6%)	2938.53 (-30%)
UpdateResource(release)	Client_x64_release.exe	14	2,322,428 K	2,244,944 K	2,930 K	212	64 bit	47	GPU 1 - 3D	1,000,992 K	697,856 K	4151.67
[D3D12-Raytracing]												
Map-V101												
GPU Upload Heaps(release)	Client_x64_release.exe	5	1,731,996 K	1,585,204 K	2,267 K	213	64 bit	99	GPU 1 - 3D	1,200,832 K	572,224 K (-8%)	440.8 (-10.5%)
UpdateResource(release)	Client_x64_release.exe	5	1,781,976 K	1,635,596 K	2,328 K	212	64 bit	99	GPU 1 - 3D	1,198,980 K	618,320 K	493.04
Map-V111												
GPU Upload Heaps(release)	Client_x64_release.exe	11	2,194,496 K	2,048,108 K	2,842 K	213	64 bit	97	GPU 1 - 3D	2,443,120 K	745,908 K (-5%)	2382.58 (-22%)
UpdateResource(release)	Client_x64_release.exe	10	2,243,956 K	2,097,136 K	2,885 K	212	64 bit	98	GPU 1 - 3D	2,445,312 K	784,356 K	3057.07

결론

- GPU Upload Heaps를 적용하면 성능향상을 볼 수 있다.
- GPU Upload Heaps를 적용하면 System Memory(Upload Buffer)사용량을 줄일 수 있다.
- GPU Upload Heaps를 적용하면 코드를 단순화 시킬 수 있다(10년 뒤에....).
- 지원하지 않는 장비들이 아직 많이 있다. 모든 상황에서 GPU Upload Heaps를 적용할 수 있지는 않다. 비동기 리소스 업데이트 자료구조는 여전히 필요하다.

참고자료

- GPU Upload Heaps
 - <https://microsoft.github.io/DirectX-Specs/d3d/D3D12GPUUploadHeaps.html>
- GPUMMU
 - <https://learn.microsoft.com/en-us/windows-hardware/drivers/display/gpu-virtual-memory-in-wddm-2-0>
- IOMMU
 - <https://learn.microsoft.com/ko-kr/windows-hardware/drivers/display/iommu-model>
- PCI Configuration space
 - https://en.wikipedia.org/wiki/PCI_configuration_space
- pcie configuration space base address register (BAR) detailed explanation
 - <https://www.programmersought.com/article/58055290325/>